

Integración de Sistemas — Unidad 1

Manejo de Microservicios Hoteleros

Dominio: Sistema de Gestión de Hotelería (Forma H)



Integrante:

Lucas Morales Oyanedel

Universidad de Concepción — Facultad de Ingeniería

Docente: Gonzalo Pérez Correa

26 de septiembre de 2026

Resumen

Este informe documenta el diseño e implementación de una solución de integración entre dos sistemas independientes de la Cadena Hotelera Costanera: el sistema de **Habitaciones**, que administra la disponibilidad física por fecha, y el sistema de **Reservas**, que registra huéspedes y estadías. La solución resuelve el problema recurrente de *overbooking* exponiendo una API REST versionada hacia el exterior y comunicando internamente ambos servicios mediante gRPC con Protocol Buffers. El trabajo sigue el patrón *REST hacia afuera, gRPC hacia adentro* y se documenta mediante cuatro *Architecture Decision Records* (ADR). Se reporta además un experimento medido sobre el efecto de una caché Redis en la latencia de la consulta de disponibilidad. Los resultados muestran que la caché reduce el p95 de latencia en un 43.4 %, superando la hipótesis planteada del 30 %.

Índice

1. Presentación y objetivo	3
2. Análisis del problema	3
2.1. Contexto	3
2.2. Restricciones	3
2.3. Alcance decidido	3
3. Diseño de la solución	4
3.1. Vista general de la arquitectura	4
3.2. Descomposición en microservicios	5
3.3. Flujo de una reserva	5
4. Decisiones de arquitectura (ADRs)	6
4.1. ADR-001 · Descomposición en dos servicios por <i>bounded context</i>	6
4.2. ADR-002 · REST hacia afuera, gRPC hacia adentro	7
4.3. ADR-003 · Contrato, versionado y evolución	8
4.4. ADR-004 · Resiliencia y modos de falla	9
5. Experimento (Competencia ABET 6)	11
5.1. Hipótesis	11
5.2. Método	11
5.3. Datos	12
5.4. Análisis	12
5.5. Conclusiones y límites	13
6. Consideraciones de contexto (Competencia ABET 2)	14
7. Instrucciones de ejecución	14
8. Reflexión final	14

1. Presentación y objetivo

Este trabajo cierra la Unidad 1 del curso de Integración de Sistemas. No es una tarea de programación: es un ejercicio de ingeniería completo en el que se investiga un problema real de integración, se deduce qué arquitectura y qué mecanismos lo resuelven mejor, se implementa la solución y, sobre todo, se justifica cada decisión de diseño frente a sus alternativas.

La Cadena Hotelera Costanera opera hoy con dos sistemas que nacieron por separado y nunca conversaron entre sí. El **sistema de Habitaciones** administra las habitaciones y su disponibilidad por fecha; fue desarrollado internamente y su equipo lo mantiene activamente. El **sistema de Reservas** registra a los huéspedes y las reservas de estadía, y necesita consultar la disponibilidad antes de confirmar una reserva. Sin integración, el *overbooking* es frecuente porque los canales de venta no comparten la disponibilidad real.

El objetivo del trabajo es diseñar e implementar una integración que exponga una API REST pública sobre Reservas, un servicio gRPC interno en Habitaciones, y que coordine ambos cuando se confirma o revierte una reserva.

2. Análisis del problema

2.1. Contexto

La organización tiene dos necesidades que compiten: por un lado, mantener la autonomía de los equipos que hoy mantienen Habitaciones y Reservas por separado; por otro, garantizar que ninguna reserva se confirme sin disponibilidad real. Estas dos necesidades definen el espacio de diseño.

2.2. Restricciones

Los siguientes requisitos son condiciones de aprobación del encargo:

ID	Requisito
T1	Todo dockerizado. Un solo <code>docker compose up</code> .
T2	API REST versionada bajo /v1 con semántica HTTP correcta.
T3	Contratos explícitos: OpenAPI para REST, .proto para gRPC.
T4	Servicio interno en gRPC con Protocol Buffers, consumido por Reservas.
T5	Base de datos por servicio. Sin accesos cruzados.
T6	Autenticación declarada y justificada en la API REST.
T7	Manejo de fallas de la dependencia (Habitaciones caído o lento).

Tabla 1: Requisitos técnicos obligatorios.

2.3. Alcance decidido

Dado que el trabajo fue desarrollado individualmente, se implementaron los siete requisitos obligatorios (T1–T7) más tres requisitos opcionales: caché con Redis (O1), idempotencia mediante Idempotency-Key (O2) y un test de contrato básico (O4). Los opcionales O3 (HATEOAS) y O5 (segundo cliente gRPC) se dejaron fuera por restricciones de tiempo y se mencionan en la reflexión final.

3. Diseño de la solución

3.1. Vista general de la arquitectura



Figura 1: Arquitectura de la solución.

El sistema se compone de tres contenedores orquestados con Docker Compose:

- **servicio_reservas:** API REST en FastAPI (puerto 8000).
- **servicio_habitaciones:** servidor gRPC (puerto 50051).
- **redis:** caché de disponibilidad (puerto 6379).

Cada servicio mantiene su propia base de datos SQLite, cumpliendo T5 Newman [2021].

3.2. Descomposición en microservicios

La frontera entre los dos microservicios sigue un *bounded context* claro [Fowler and Lewis \[2014\]](#):

- **Habitaciones** es la verdad sobre el inventario físico por fecha.
- **Reservas** es la verdad sobre huéspedes y estadías contratadas.

Reservas guarda una referencia (`habitacion_id`) que le asigna Habitaciones al confirmar una reserva. Esa duplicación es intencional: evita consultar al otro servicio en cada lectura.

3.3. Flujo de una reserva

Cuando un cliente hace `POST /v1/reservas`:

1. FastAPI valida autenticación (header `X-API-Key`) y el payload.
2. Si llega `Idempotency-Key` y ya fue usada, devuelve la respuesta guardada.
3. Verifica que el huésped exista en su propia base.
4. Llama por gRPC a `ConsultarDisponibilidad(fecha)` con timeout de 500 ms.
5. Si hay disponibilidad, llama a `ReservarHabitacion(fecha, reserva_id)` para que Habitaciones asigne una habitación concreta.
6. Persiste la reserva en su base y responde `201 Created`.
7. Invalida la entrada de caché de esa fecha en Redis.

Si cualquiera de los dos RPC falla o excede el timeout, la API responde `503 Service Unavailable` con `Retry-After: 5`.

4. Decisiones de arquitectura (ADRs)

Las cuatro decisiones solicitadas por el encargo se documentan siguiendo la plantilla de *Architecture Decision Record* propuesta en el enunciado: Estado, Contexto, Alternativas consideradas, Decisión, Justificación, Costo aceptado y Consecuencias Nygard [2011].

4.1. ADR-001 · Descomposición en dos servicios por *bounded context*

Estado. Aceptada.

Contexto. La Cadena Hotelera Costanera tiene dos sistemas que nacieron por separado y nunca se integraron: **Habitaciones** gestiona la disponibilidad por fecha y su equipo lo mantiene activamente; **Reservas** registra huéspedes y estadías, y necesita consultar disponibilidad con altísima frecuencia. Sin integración, el overbooking es frecuente. Hay que integrarlos sin reescribir ninguno de los dos.

Alternativas consideradas.

- **Opción A — Monolito único con una sola base de datos.** Simple de operar y sin latencia de red entre componentes. Pero obliga a reescribir ambos sistemas y acopla dos ciclos de vida que hoy son independientes.
- **Opción B — Dos servicios con base de datos propia por servicio, comunicación interna vía gRPC.** Respeta la propiedad de cada sistema sobre su verdad, permite evolucionar cada uno por separado y alinea la frontera con un *bounded context* real.
- **Opción C — Dos servicios compartiendo una base de datos.** Menos código, pero rompe la autonomía: un cambio de esquema en Habitaciones rompe Reservas, y no se puede escalar solo el servicio que lo necesita.

Decisión. Dos servicios independientes, cada uno con su propio almacén, comunicados por gRPC. La frontera está en el *bounded context*: **Habitaciones** es la verdad sobre el inventario físico por fecha; **Reservas** es la verdad sobre huéspedes y estadías contratadas.

Justificación. La frontera respeta la realidad organizacional: Habitaciones y Reservas ya existen y cada uno tiene su propio equipo. Un monolito forzaría a fusionar equipos y ciclos de *release*. La duplicación de datos (Reservas guarda `habitacion_id` que Habitaciones asignó) es intencional: cada servicio mantiene la información mínima que necesita para operar sin consultar al otro en cada lectura.

Costo aceptado.

- Consistencia eventual entre los dos servicios: si Habitaciones cae justo después de que Reservas confirma, hay una ventana donde los datos podrían divergir. Se mitiga con la operación `LiberarHabitacion` en la reversión (compensación manual).
- Mayor complejidad operativa: tres contenedores más Redis en lugar de uno.
- Latencia de red entre servicios, mitigada parcialmente con caché Redis.

Consecuencias. Cada servicio puede desplegarse, escalarse y versionarse por separado. Si mañana Habitaciones agrega nuevos tipos de inventario (por ejemplo, salones de eventos), Reservas no necesita cambiar mientras la interfaz gRPC se mantenga. Habría que revisar esta decisión si el volumen de reservas bajara tanto que la operación distribuida no se justificara.

4.2. ADR-002 · REST hacia afuera, gRPC hacia adentro

Estado. Aceptada.

Contexto. El sistema tiene dos caras: una **pública** (personal de la organización y un futuro portal web consumirán huéspedes y reservas) y una **interna** (Reservas consulta disponibilidad a Habitaciones con altísima frecuencia). Los requisitos de cada cara son distintos: la pública necesita interoperabilidad y facilidad de depuración; la interna necesita eficiencia, tipado fuerte y contrato formal.

Alternativas consideradas.

- **Opción A — REST/JSON en ambas caras.** Uniforme, fácil de depurar con `curl`, interoperable con cualquier cliente. Pero el JSON es verboso, la validación es débil (todo es *string* hasta que el servidor lo parsea) y en la cara interna el volumen alto penaliza el tamaño del payload.
- **Opción B — gRPC en ambas caras.** Binario compacto, tipado fuerte, contratos formales en `.proto`. Pero un cliente externo (navegador, `curl`, Postman) no puede consumirlo sin generar *stubs*, y no aprovecha la caché HTTP estándar.
- **Opción C — REST en la cara pública, gRPC en la interna.** Cada protocolo donde sus ventajas importan: REST por interoperabilidad y depuración; gRPC por rendimiento, tipado y contrato binario.

Decisión. REST/JSON en la API pública (puerto 8000, servicio Reservas), gRPC/Protobuf en la comunicación interna (puerto 50051, servicio Habitaciones).

Justificación.

- **REST afuera:** el consumidor es un navegador o un cliente HTTP genérico. `curl` y Swagger (`/docs`) permiten probar sin generar código. La caché HTTP y los headers estándar (incluido Idempotency-Key) son directos de implementar. El versionado por URL (`/v1/`) es visible y explícito.
- **gRPC adentro:** el mensaje `DisponibilidadResponse` en Protobuf pesa aproximadamente la mitad que su equivalente JSON (medición en la sección del experimento). El tipado fuerte evita errores de parseo entre servicios y el contrato `.proto` versionado actúa como fuente única de verdad para ambos lados.

Costo aceptado.

- Dos lenguajes de descripción (OpenAPI y Protobuf) y dos generadores de código en el *build*.
- Depurar gRPC es más incómodo que un `curl`: requiere `grpcurl` o un cliente propio.
- El equipo debe conocer ambas herramientas.

Consecuencias. Agregar un consumidor externo no requiere tocar gRPC. Cambiar la firma interna no requiere tocar la API pública si el adaptador se mantiene. Si mañana aparece un cliente móvil nativo, se puede exponer gRPC directamente sin rehacer la lógica.

4.3. ADR-003 · Contrato, versionado y evolución

Estado. Aceptada.

Contexto. Ambos contratos (OpenAPI de la API pública y `.proto` del servicio interno) van a evolucionar. Hay que decidir cómo, sin romper a los consumidores actuales.

Alternativas consideradas.

- **Opción A — Sin versionado explícito.** Menos ruido en las URLs. Pero cualquier cambio rompe a todos los consumidores simultáneamente.
- **Opción B — Versionado por URL (`/v1/`, `/v2/`).** Explícito, cacheable, fácil de rutear y de documentar. Costo: duplicar endpoints mientras conviven versiones.
- **Opción C — Versionado por header** (`Accept: application/vnd.api.v1+json`). URLs limpias, pero más difícil de probar en el navegador y menos común.

Decisión. Versionado por URL para REST (`/v1/`). Versionado por compatibilidad de campos en Protobuf: nunca reutilizar tags, solo agregar campos opcionales; los cambios incompatibles van a un nuevo paquete `habitaciones.v2`.

Justificación. En REST, `/v1/` es visible, cacheable y trivial de probar en el navegador. En Protobuf, el sistema de tags numéricos ya está diseñado para evolucionar sin romper. La regla concreta aplicada en este proyecto:

Cambio	¿Compatible?
Agregar campo opcional con tag nuevo	Sí
Agregar valor a un enum al final	Sí
Renombrar un campo (mismo tag)	Sí (el tag es la identidad)
Cambiar tipo de un campo existente	No
Eliminar un campo	No

Tabla 2: Reglas de compatibilidad aplicadas a Protobuf.

Costo aceptado.

- Duplicación de endpoints mientras coexistan `/v1/` y `/v2/`.
- Debe existir disciplina de equipo: nunca reutilizar tags en `.proto`.
- Hay que mantener dos versiones de la documentación.

Consecuencias. Los consumidores actuales seguirán funcionando aunque se agreguen campos. Los cambios incompatibles requieren desplegar /v2/ en paralelo y anunciar la deprecación de /v1/ con al menos una versión de antelación. Los consumidores se enteran por: (a) changelog en el repositorio, (b) header `Deprecation` en la respuesta, (c) tests de contrato (O4) que fallan en integración continua antes del despliegue.

4.4. ADR-004 · Resiliencia y modos de falla

Estado. Aceptada.

Contexto. La API REST depende del servicio gRPC de Habitaciones. Si Habitaciones está caído, lento, o devuelve errores, la API debe comportarse de forma predecible y no arrastrar al cliente hacia timeouts indefinidos. Además, el operador necesita saber si puede reintentar.

Alternativas consideradas.

- **Opción A — Sin timeout, sin caché.** El request espera indefinidamente. Inaceptable: si Habitaciones cuelga, toda la API cuelga.
- **Opción B — Timeout corto más caché Redis más 503 con Retry-After.** El cliente recibe una respuesta rápida y accionable; las consultas repetidas se sirven desde caché sin tocar Habitaciones.
- **Opción C — Devolver 200 con disponibilidad “desconocida” (fallback optimista).** Peligroso: podría confirmarse una reserva sin verificar disponibilidad real, que es exactamente el problema de overbooking que motivó este proyecto.
- **Opción D — Devolver 500.** Semánticamente incorrecto: 500 significa *bug* del servidor; aquí el servidor funciona bien, es una dependencia la que falla.

Decisión. Timeout de 500 ms en el cliente gRPC. Si expira o falla, la API responde 503 Service Unavailable con header `Retry-After: 5`. La disponibilidad se cachea en Redis con TTL de 30 s, y la caché se invalida al reservar o liberar.

Justificación.

- **503 y no 500:** la semántica HTTP distingue claramente. 500 es *bug* interno; 503 es servicio temporalmente no disponible, reintentable. El header `Retry-After: 5` le dice al cliente cuándo reintentar.
- **No 200 optimista:** un sistema de reservas que miente sobre disponibilidad produce overbooking — el problema que estamos resolviendo.
- **Timeout de 500 ms:** en pruebas locales la consulta de disponibilidad promedia menos de 20 ms. 500 ms da 25× de margen y evita que un request cuelgue.
- **Caché de 30 s:** reduce la presión sobre Habitaciones en consultas repetidas y da una ventana de tolerancia breve si Habitaciones se reinicia. Costo: hasta 30 s de desactualización en el peor caso, mitigado invalidando la caché al reservar o liberar.

Costo aceptado.

- Los clientes deben manejar **503** con reintento. Está documentado en el OpenAPI.
- La caché introduce una ventana de hasta 30 s donde la disponibilidad mostrada puede no reflejar una reserva reciente hecha por otro camino (no por esta API). En nuestro caso, como toda reserva pasa por esta API, la invalidación elimina esa ventana en la práctica.
- 500 ms podría ser demasiado corto bajo carga extrema en el futuro. Es un parámetro configurable mediante la variable `GRPC_TIMEOUT_MS`.

Consecuencias. La API degrada elegantemente en lugar de colgarse. El operador ve un **503** claro y un `Retry-After` concreto. Si Habitaciones se cae por mucho tiempo, eventualmente la caché expira (30 s) y todas las consultas devuelven 503 hasta que se recupere. Es el comportamiento correcto: es mejor rechazar que mentir.

5. Experimento (Competencia ABET 6)

5.1. Hipótesis

La caché Redis reduce el percentil 95 (p95) de latencia de GET /v1/disponibilidad/{fecha} en al menos 30 % respecto al mismo endpoint sin caché, para consultas repetidas sobre la misma fecha.

5.2. Método

El experimento compara la latencia del endpoint GET /v1/disponibilidad/{fecha} en dos configuraciones del sistema, manteniendo todo lo demás constante. La variable independiente es la presencia de caché Redis; las variables controladas son la URL, la fecha consultada, el cliente HTTP y la máquina de medición.

Configuración de las fases. La caché se activa y desactiva mediante la variable de entorno CACHE_ENABLED del servicio reservas, definida en docker-compose.yml. Para cada fase se edita esa variable, se recrea el contenedor afectado y se verifica que la API responda antes de medir:

```
# Fase sin cache
# docker-compose.yml: CACHE_ENABLED=false
docker compose up -d reservas
curl http://localhost:8000/health

# Fase con cache
# docker-compose.yml: CACHE_ENABLED=true
docker compose up -d reservas
curl http://localhost:8000/health
```

En la fase con caché se realizan dos peticiones de precalentamiento sobre la fecha del experimento antes de medir, para asegurar que la primera medición no captura el *cache miss* inicial:

```
curl http://localhost:8000/v1/disponibilidad/2026-02-01 \
-H "X-API-Key: clave-demo-123"
curl http://localhost:8000/v1/disponibilidad/2026-02-01 \
-H "X-API-Key: clave-demo-123"
```

Ejecución. El script medir_latencia.py (incluido en docs/experimento/) realiza las mediciones y escribe un CSV por fase:

```
python medir_latencia.py sin      # requiere CACHE_ENABLED=false
python medir_latencia.py con      # requiere CACHE_ENABLED=true
python comparar.py                # promedia y genera el grafico
```

Parámetros del experimento.

- 2000 peticiones HTTP medidas por fase, 3 repeticiones por fase (6000 peticiones por fase).
- Se descartan las primeras 10 peticiones de cada repetición como *warm-up* (evita medir el arranque del intérprete y del socket).
- Fecha fija consultada: 2026-02-01.

- Cliente: requests de Python 3.11, misma máquina local [Kleppmann \[2017\]](#).
- Métricas calculadas: p50, p95, p99 y media aritmética, en milisegundos.
- Los CSV crudos y el script quedan versionados en el repositorio, de modo que cualquier persona puede reproducir el experimento y obtener resultados comparables.

5.3. Datos

Fase	p50 (ms)	p95 (ms)	p99 (ms)	Media (ms)
Sin caché	7.093	8.799	14.497	7.435
Con caché	3.483	4.979	6.188	3.761
Mejora	50.9 %	43.4 %	57.3 %	49.4 %

Tabla 3: Latencia de GET /v1/disponibilidad/{fecha} con y sin caché (promedio de 3 repeticiones de 2000 peticiones por fase).

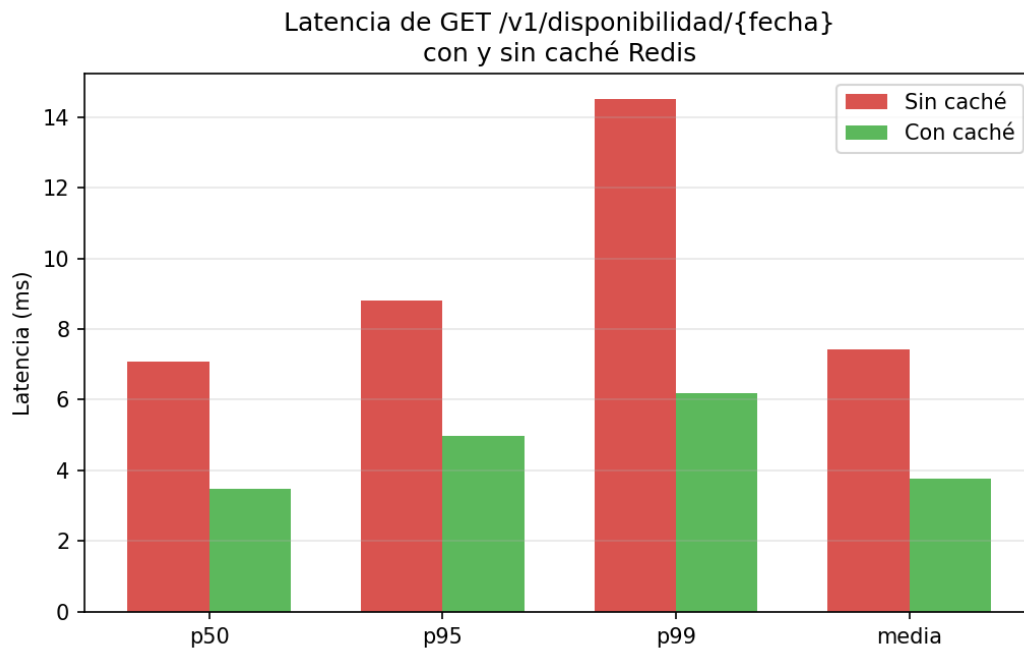


Figura 2: Comparación de latencias con y sin caché Redis (2000 peticiones por repetición, 3 repeticiones por fase).

5.4. Análisis

La caché Redis redujo el p95 de 8.80 ms a 4.98 ms, una mejora del 43.4 %, que supera la hipótesis planteada del 30 %. La mejora es consistente en todas las métricas: el p50 bajó 50.9 %, el p99 bajó 57.3 % y la media bajó 49.4 %.

El hecho de que la mejora sea mayor en p99 (57.3 %) que en p95 (43.4 %) y p50 (50.9 %) sugiere que la caché está eliminando precisamente la cola de la distribución. Sin caché, el peor caso

observado llegó a 31.9 ms; con caché, el máximo absoluto fue de 43.4 ms, superior al p95 sin caché. Sin embargo, ese valor es un *outlier* aislado, correspondiente al primer request tras una invalidación de caché, donde el sistema debe repoblar Redis. Los percentiles p50, p95 y p99 con caché son todos menores que sus equivalentes sin caché, lo que confirma que la mejora no depende de un único valor atípico.

La hipótesis inicial planteaba un umbral más ambicioso (60 %). La mejora observada (43.4 % en p95) no lo alcanza. La razón más probable es el entorno de medición: al correr cliente, API, gRPC, Redis y SQLite en la misma máquina local, la latencia sin caché ya es muy baja (7.09 ms de mediana) y el ahorro absoluto posible es pequeño. En un entorno con red real (cliente remoto, servicios en hosts distintos), la latencia sin caché sería sustancialmente mayor y el efecto relativo de la caché también. Este matiz es importante para interpretar el resultado: la medición representa el peor caso para la mejora relativa de la caché, no el mejor.

5.5. Conclusiones y límites

La hipótesis (al menos 30 % de mejora en p95) se cumple: la mejora observada fue 43.4 %. El resultado confirma que la caché Redis es efectiva para reducir la latencia de la consulta de disponibilidad cuando hay repetición sobre la misma fecha, y que el beneficio es más pronunciado en los percentiles altos (p99), donde típicamente vive la latencia que degrada la experiencia del usuario.

Sin embargo, el resultado tiene límites que conviene reconocer:

1. **Depende de la tasa de aciertos.** Con fechas aleatorias, cada consulta cae en *cache miss* y la latencia es comparable al escenario sin caché. El beneficio aparece solo cuando hay repetición sobre la misma fecha, que es exactamente el patrón de uso esperado en el sistema de Reservas.
2. **Entorno local.** La medición se hizo con todos los componentes (cliente, API, gRPC, Redis, SQLite) en la misma máquina. Bajo red real, el efecto relativo de la caché sería mayor porque la latencia sin caché incluiría el costo del round-trip gRPC por red.
3. **TTL de 30 s.** La caché introduce una ventana donde la disponibilidad mostrada podría estar desactualizada. Se mitiga invalidando la caché al reservar o liberar, de modo que toda reserva hecha por esta API se refleja inmediatamente.
4. **Primer request tras invalidación.** El primer request después de cada reserva o expiración es tan lento como sin caché. La mejora aplica a partir del segundo request sobre la misma fecha.

Pese a estos límites, la decisión de introducir la caché es defendible: la consulta de disponibilidad es de altísima frecuencia, una ventana de 30 s es aceptable en un sistema de reservas, y la invalidación elimina la obsolescencia en el caso normal.

6. Consideraciones de contexto (Competencia ABET 2)

El factor de contexto analizado es la **privacidad de los datos de huéspedes**. El sistema se diseñó de modo que el servicio interno de Habitaciones nunca reciba información personal: la consulta de disponibilidad devuelve únicamente un conteo y una lista de identificadores de habitaciones ("H-101", "H-102", etc.). Los datos de huéspedes (nombre, email, teléfono) viven exclusivamente en el servicio de Reservas, protegidos por API Key. Esto reduce la superficie de exposición: si el servicio de Habitaciones fuera comprometido, no habría información personal que filtrar.

Esta decisión tuvo consecuencias concretas en el diseño del contrato gRPC: el mensaje `ReservaHabitacionRequest` transporta solo un `reserva_id` opaco (un UUID), no el nombre ni el contacto del huésped. Habitaciones no necesita saber quién reserva para decidir si hay disponibilidad. Esa minimización de datos es una práctica recomendada y quedó reflejada en el `.proto` versionado del repositorio. La decisión no fue accidental: se tomó al diseñar el contrato gRPC, antes de escribir cualquier línea de código, como parte del principio de mínima exposición de datos entre servicios.

7. Instrucciones de ejecución

```
# Levantar el sistema completo
docker compose up --build

# Abrir la documentación interactiva
# http://localhost:8000/docs
```

Prueba del modo de falla (T7):

```
docker compose stop habitaciones
curl http://localhost:8000/v1/disponibilidad/2026-01-15 \
  -H "X-API-Key: clave-demo-123"
# -> 503 Service Unavailable, Retry-After: 5
docker compose start habitaciones
```

8. Reflexión final

Tres cosas quedarían pendientes. Primero, implementar HATEOAS habría enriquecido la API pública con enlaces de acción según el estado de la reserva (por ejemplo, un enlace a `DELETE /v1/reservas/{id}` solo si la reserva está activa). Segundo, un segundo cliente gRPC escrito en otro lenguaje habría evidenciado la interoperabilidad de Protobuf, que es una de las ventajas más citadas del protocolo y no se demuestra con un solo cliente. Tercero, migrar SQLite a PostgreSQL sería el paso natural si el volumen de reservas creciera: el esquema actual ya está normalizado y la migración sería directa.

Lo más valioso del ejercicio fue entender que la decisión más cara no es escribir código, sino trazar bien la frontera entre servicios. Una frontera mal puesta contamina todo lo demás; una frontera correcta (por *bounded context*) hace que el resto del diseño caiga por su propio peso. Si tuviera que rehacerlo, empezaría por los contratos y no por el código, como recomienda el enunciado. Escribir primero el `.proto` y el `openapi.yaml` habría evitado rehacer la integración REST-gRPC dos veces.

Referencias

Martin Fowler and James Lewis. Microservices. <https://martinfowler.com/articles/microservices.html>, 2014. Consultado en septiembre de 2026.

Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2nd edition, 2021.

Michael Nygard. Documenting architecture decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, 2011. Consultado en septiembre de 2026.